

Лабораторная работа № 5.

Подготовка приложения к Production-окружению

Тема: Environment configuration, health checks, graceful shutdown и production readiness

Цель: Подготовить Internet Shop API к production-запуску: вынести configuration в environment variables, реализовать fail-fast startup validation, liveness/readiness endpoints, graceful shutdown HTTP server и Prisma, безопасную обработку ошибок и настройки работы за reverse проху.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект `express-shop`. Должны работать Express, Prisma/PostgreSQL, JWT, validation, Pino logging и integration tests с Supertest. Docker в этой лабораторной пока не используется.

1. Проведите аудит текущей configuration

1. Найдите все значения, которые зависят от environment.
2. Проверьте PORT.
3. Проверьте DATABASE_URL.
4. Проверьте JWT_SECRET.
5. Проверьте CORS origin.
6. Проверьте LOG_LEVEL.
7. Убедитесь, что production secrets не записаны literal-строками в JavaScript.

2. Создайте пример production environment

```
.env.production.example

NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://app_user:CHANGE_ME@localhost:5432/shop
JWT_SECRET=CHANGE_ME
CORS_ORIGIN=https://shop.example.com
LOG_LEVEL=info
```

Файл `.env.production.example` содержит только пример имён variables. Реальные secrets в него не записываются.

3. Проверьте .gitignore

```
.gitignore

.env
.env.*
!.env.example
!.env.production.example
node_modules/
logs/
```

Реальный `.env.production` не должен попадать в Git.

4. Создайте config/env.js

```
config/env.js

function requiredEnv(name) {
  const value = process.env[name];

  if (!value) {
    throw new Error(
      `Missing required environment variable: ${name}`
    );
  }

  return value;
}

const port = Number(process.env.PORT ?? 3000);

if (!Number.isInteger(port) || port < 1 || port > 65535) {
  throw new Error("PORT must be an integer from 1 to 65535");
}

export const env = {
  nodeEnv: process.env.NODE_ENV ?? "development",
  port,
  databaseUrl: requiredEnv("DATABASE_URL"),
  jwtSecret: requiredEnv("JWT_SECRET"),
  corsOrigin: requiredEnv("CORS_ORIGIN"),
  logLevel: process.env.LOG_LEVEL ?? "info"
};
```

5. Используйте единый config object

Замените прямое чтение `process.env.PORT`, `process.env.JWT_SECRET` и других обязательных variables в разных файлах проекта на импорт из `config/env.js`, где это возможно.

Пример

```
import { env } from "../config/env.js";

console.log(env.nodeEnv);
console.log(env.port);
```

Не выводите `env.jwtSecret` и `env.databaseUrl` в logs.

6. Проверьте fail-fast startup

8. Временно не задавайте `JWT_SECRET`.
9. Запустите приложение.
10. Убедитесь, что startup завершается понятной ошибкой `Missing required environment variable: JWT_SECRET`.
11. Верните `JWT_SECRET`.
12. Повторно запустите приложение.

7. Разделите app.js и server.js

```
server.js
```

```
import { app } from "../app.js";
import { env } from "../config/env.js";
import { logger } from "../config/logger.js";

export const server = app.listen(env.port, () => {
  logger.info({
    port: env.port,
    nodeEnv: env.nodeEnv
  }, "HTTP server started");
});
```

`app.js` не должен вызывать `app.listen()`. Это позволяет использовать `request(app)` в integration tests.

8. Добавьте production npm scripts

```
package.json

{
  "scripts": {
    "start": "node server.js",
    "start:prod:local": "node --env-file=.env.production server.js",
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js"
  }
}
```

`start:prod:local` нужен только для локальной имитации production. В реальном deployment environment variables обычно передаются инфраструктурой.

9. Создайте локальный .env.production

```
.env.production

NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://shop_user:college123@localhost:5432/shop_db
JWT_SECRET=replace-with-local-test-secret
CORS_ORIGIN=http://localhost:5500
LOG_LEVEL=info
```

Этот файл используется только локально и должен оставаться в `.gitignore`.

10. Добавьте liveness endpoint

```
routes/healthRoutes.js

import { Router } from "express";
import { prisma } from "../config/prisma.js";

export const healthRoutes = Router();

healthRoutes.get("/live", (req, res) => {
  res.status(200).json({
    status: "ok"
  });
});
```

Liveness не должен выполнять тяжёлые запросы. Его задача — показать, что process жив и HTTP application отвечает.

11. Добавьте readiness endpoint

```
routes/healthRoutes.js

healthRoutes.get("/ready", async (req, res) => {
  try {
    await prisma.$queryRaw`SELECT 1`;

    res.status(200).json({
      status: "ready"
    });
  } catch (error) {
    req.log?.warn({ err: error }, "Readiness check failed");

    res.status(503).json({
      status: "not_ready"
    });
  }
});
```

12. Подключите health routes

```
app.js

import { healthRoutes } from "../routes/healthRoutes.js";

app.use("/health", healthRoutes);
```

Проверяемые URL

```
GET http://localhost:3000/health/live
GET http://localhost:3000/health/ready
```

13. Проверьте liveness и readiness

13. Запустите PostgreSQL и приложение.
14. Проверьте `/health/live` → 200.
15. Проверьте `/health/ready` → 200.
16. Остановите PostgreSQL.
17. Проверьте `/health/live` → приложение всё ещё должно отвечать.
18. Проверьте `/health/ready` → 503.
19. Запустите PostgreSQL снова и убедитесь, что readiness вернулся к 200.

14. Добавьте integration tests для health endpoints

```
tests/integration/health.api.test.js

import request from "supertest";
import { describe, expect, test } from "@jest/globals";
import { app } from "../../app.js";
```

```
describe("Health API", () => {
  test("GET /health/live returns 200", async () => {
    const response = await request(app)
      .get("/health/live");

    expect(response.status).toBe(200);
    expect(response.body).toEqual({
      status: "ok"
    });
  });

  test("GET /health/ready returns 200 when DB is ready", async () => {
    const response = await request(app)
      .get("/health/ready");

    expect(response.status).toBe(200);
    expect(response.body).toEqual({
      status: "ready"
    });
  });
});
```

15. Реализуйте graceful shutdown

```
server.js

import { prisma } from "./config/prisma.js";

let shuttingDown = false;

async function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;

  logger.info({ signal }, "Shutdown started");

  server.close(async (error) => {
    if (error) {
      logger.error({ err: error }, "HTTP server close failed");
      process.exitCode = 1;
      return;
    }

    await prisma.$disconnect();
    logger.info("Shutdown completed");
  });
}

process.once("SIGTERM", () => shutdown("SIGTERM"));
process.once("SIGINT", () => shutdown("SIGINT"));
```

16. Добавьте force timeout

```
server.js

const shutdownTimeout = setTimeout(() => {
  logger.fatal("Graceful shutdown timeout");
```

```
server.closeAllConnections();
process.exit(1);
}, 10000);

shutdownTimeout.unref();
```

Перенесите создание этого timer внутрь `shutdown()`, чтобы он запускался только при начале завершения `process`.

17. Исправьте shutdown timeout

Фрагмент `shutdown()`

```
const forceTimer = setTimeout(() => {
  logger.fatal("Graceful shutdown timeout");
  server.closeAllConnections();
  process.exit(1);
}, 10000);

forceTimer.unref();
```

После успешного закрытия `server` и `Prisma` можно выполнить `clearTimeout(forceTimer)`.

18. Проверьте graceful shutdown

20. Запустите приложение.
21. Выполните несколько HTTP requests.
22. Нажмите Ctrl+C.
23. Убедитесь, что в log появился `Shutdown started`.
24. Убедитесь, что HTTP server закрыт.
25. Убедитесь, что Prisma отключена.
26. Убедитесь, что process завершился без зависания.

19. Настройте production-safe error response

`middleware/errorHandler.js`

```
export function errorHandler(error, req, res, next) {
  req.log?.error({
    err: error,
    method: req.method,
    path: req.originalUrl
  }, "Request failed");

  const status = error.status ?? 500;

  res.status(status).json({
    error: status < 500
      ? error.message
      : "Internal server error",
    requestId: req.requestId
  });
}
```

Unexpected 500 не должен возвращать stack trace клиенту.

20. Отключите X-Powered-By

```
app.js  
  
app.disable("x-powered-by");
```

Это уменьшает лишнее раскрытие информации о framework, хотя само по себе не является полноценной защитой.

21. Настройте trust proxy только для известной topology

```
app.js  
  
import { env } from "../config/env.js";  
  
if (env.nodeEnv === "production") {  
  app.set("trust proxy", 1);  
}
```

Значение `1` означает доверие одному проху hop. Не копируйте эту настройку в реальный deployment без проверки topology reverse proxy.

22. Проверьте production startup

```
PowerShell  
  
npm run start:prod:local
```

- 27. Проверьте, что `NODE_ENV=production`.
- 28. Проверьте startup logs.
- 29. Убедитесь, что secrets не выводятся.
- 30. Проверьте health endpoints.
- 31. Проверьте один public endpoint.
- 32. Проверьте один protected endpoint.

23. Запустите regression suite

```
PowerShell  
  
npm test
```

Production preparation не должна ломать unit и integration tests.

24. Самостоятельное задание

- 33. Добавьте validation для `NODE_ENV`: разрешите только development, test и production.
- 34. Добавьте startup check для `CORS_ORIGIN`.
- 35. Добавьте integration test заголовка X-Request-Id.
- 36. Добавьте test, что `/health/live` не раскрывает configuration.
- 37. Добавьте test, что `/health/ready` возвращает JSON.
- 38. Сымитируйте отсутствие `JWT_SECRET` и зафиксируйте fail-fast behavior.
- 39. Проверьте Ctrl+C shutdown.
- 40. Убедитесь, что Prisma disconnect вызывается при shutdown.

41. Проверьте, что `stack trace` не возвращается клиенту.
42. Составьте собственный `production-readiness checklist` проекта.

Требования к результату

- Создан строгий `config/env.js`.
- Обязательные `environment variables` проверяются при `startup`.
- `Secrets` отсутствуют в исходном коде и `Git`.
- `app.js` отделён от `server.js`.
- `NODE_ENV=production` используется при `production startup`.
- Есть `/health/live`.
- Есть `/health/ready`.
- `Readiness` проверяет `PostgreSQL`.
- Реализованы `SIGTERM/SIGINT handlers`.
- `HTTP server` закрывается через `server.close()`.
- `Prisma` отключается через `prisma.$disconnect()`.
- Есть `shutdown timeout`.
- Настроен `production-safe error response`.
- `trust proxy` описан и используется только для известной `topology`.
- Все `unit/integration tests` проходят.

Критерии оценивания

№	Критерий
1	Создан <code>config/env.js</code> с <code>required variables</code>
2	<code>PORT</code> валидируется
3	<code>Secrets</code> не хранятся <code>literal</code> в коде
4	Есть <code>.env.production.example</code> без реальных <code>secrets</code>
5	<code>app.js</code> и <code>server.js</code> разделены
6	<code>/health/live</code> возвращает <code>200</code>
7	<code>/health/ready</code> проверяет <code>DB</code>
8	При проблеме <code>DB readiness</code> возвращает <code>503</code>
9	Обрабатывается <code>SIGINT</code>
10	Обрабатывается <code>SIGTERM</code>
11	<code>server.close()</code> используется при <code>shutdown</code>
12	<code>Prisma</code> отключается при <code>shutdown</code>
13	Есть <code>force shutdown timeout</code>
14	<code>Production error response</code> не раскрывает <code>stack</code>
15	Студент объясняет назначение <code>reverse proxy</code> , <code>trust proxy</code> , <code>liveness</code> и <code>readiness</code>

